

André Barbosa Alves dos Santos
Rafael Marques Mansano
Vinicius de Oliveira Silva

Agente de Análise Exploratória de Dados em Linguagem Natural com Tool Use

Ourinhos

2026

1 Introdução

A análise exploratória de dados (EDA) é uma etapa essencial em projetos de ciência de dados, pois permite compreender a estrutura de um conjunto de informações, identificar padrões, detectar inconsistências e gerar hipóteses para análises posteriores. Entretanto, a execução dessas tarefas normalmente exige conhecimentos de programação e domínio de bibliotecas especializadas, o que pode representar uma barreira para usuários sem experiência técnica.

Com o avanço dos modelos de linguagem de grande porte (LLMs), tornou-se possível utilizar a linguagem natural como forma de interação com sistemas computacionais. Nesse contexto, agentes baseados em LLMs podem interpretar perguntas formuladas em português, selecionar ferramentas adequadas e executar operações sobre dados reais, tornando processos analíticos mais acessíveis e intuitivos.

Este trabalho teve como objetivo desenvolver um agente de análise exploratória de dados capaz de responder perguntas em linguagem natural sobre um arquivo CSV. Para isso, foi utilizada a API *DeepSeek* integrada a um conjunto de ferramentas implementadas em *Python*, responsáveis por executar operações como filtragem, agrupamentos, estatísticas descritivas, correlações, detecção de outliers e geração de gráficos.

A utilização de ferramentas especializadas permite que os cálculos sejam realizados diretamente sobre o *dataset*, reduzindo a dependência do conhecimento paramétrico do modelo e diminuindo o risco de respostas incorretas ou alucinações. Dessa forma, o agente atua como um intermediário entre o usuário e os dados, combinando compreensão de linguagem natural com processamento determinístico.

Para avaliar o sistema, foi desenvolvido um benchmark composto por 30 perguntas factuais, analíticas e ambíguas. O agente foi avaliado por meio de métricas como acurácia, taxa de execução, número médio de chamadas de ferramentas, latência e custo estimado. Este relatório apresenta a arquitetura do sistema, as decisões de implementação, os resultados obtidos e uma análise crítica das dificuldades encontradas durante o desenvolvimento.

2 Descrição do Dataset

O *dataset* utilizado no projeto representa vendas de *e-commerce*. Ele foi escolhido por possuir uma estrutura adequada para análises exploratórias, com variáveis numéricas, variáveis categóricas, registros suficientes para agregações e campos relacionados a pedidos, clientes e valores financeiros.

O arquivo utilizado foi `data/vendas_ecommerce.csv`, contendo 1.500 registros e 10 colunas. A Tabela 1 apresenta as colunas do *dataset*.

Tabela 1 – Colunas do dataset de vendas de *e-commerce*

Coluna	Descrição geral
pedido_id	Identificador do pedido
idade_cliente	Idade do cliente
cidade	Cidade ou estado associado ao pedido
categoria	Categoria do produto
tipo_cliente	Tipo do cliente
forma_pagamento	Forma de pagamento utilizada
quantidade	Quantidade de itens no pedido
preco_unitario	Preço unitário do item
desconto	Desconto aplicado
valor_total	Valor total do pedido

Fonte: Elaborado pelos autores (2026).

O *dataset* atende aos requisitos do trabalho por possuir mais de 1.000 registros, ao menos 6 colunas e uma combinação de variáveis numéricas e categóricas. As variáveis numéricas permitem análises estatísticas, correlações, percentis e detecção de outliers. Já as variáveis categóricas permitem contagens, agrupamentos, filtros e análise de distribuição por grupos.

Durante a avaliação do agente, algumas perguntas exploraram características simples do *dataset*, como número de linhas e colunas, enquanto outras exigiram operações mais complexas, como cálculo de média por grupo, soma por categoria, correlação entre variáveis e detecção de *outliers*.

3 Arquitetura do Agente

A arquitetura do sistema foi organizada em camadas, separando interface, orquestração, ferramentas e dados. Essa divisão facilitou a implementação, os testes e a depuração.

3.1 Visão Geral da Arquitetura

A Figura apresenta uma visão simplificada do funcionamento do agente.

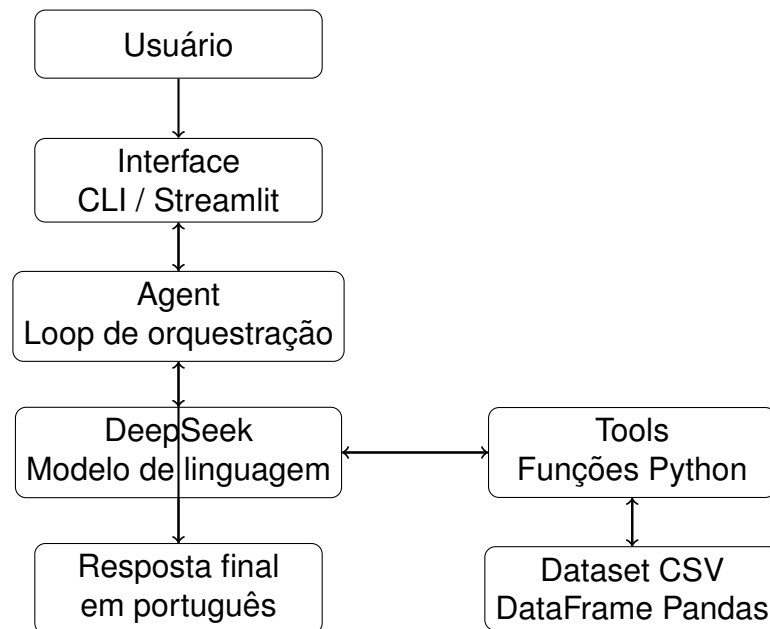


Figura 1 – Arquitetura simplificada do agente de análise exploratória de dados

Fonte: Elaborado pelos autores (2026).

O usuário interage com o sistema por meio da CLI ou da interface *Streamlit*. A pergunta é encaminhada para o agente, que envia ao modelo *DeepSeek* o histórico da conversa, o prompt de sistema e a descrição das ferramentas disponíveis. Quando o modelo solicita uma ferramenta, o agente executa a função Python correspondente, registra a trajetória e devolve o resultado ao modelo. O ciclo continua até que o modelo produza uma resposta final.

3.2 Camada de Interface

Foram implementadas duas formas principais de interação: CLI e Streamlit. A CLI permite execução em terminal, enquanto o *Streamlit* oferece uma interface web mais próxima de aplicações de IA generativa, com campo de entrada de texto, respostas em *Markdown*, *upload* de CSV, visualização do *dataset*, execução do *benchmark* e exibição de gráficos.

A interface *Streamlit* foi importante para tornar o projeto mais acessível, pois permite ao usuário interagir com o agente sem utilizar comandos no terminal. Além disso, a interface exibe métricas como número de chamadas de ferramentas, latência, tokens e custo estimado.

3.3 Camada de Orquestração

A camada de orquestração é implementada principalmente no arquivo `agent/agent.py`. Ela é responsável por controlar o ciclo de execução do agente, armazenar a trajetória, executar ferramentas solicitadas e acumular métricas. Essa camada também define o prompt de sistema, que orienta o comportamento do agente.

O cliente de LLM foi implementado no arquivo `agent/llm_client.py`. Ele adapta a comunicação com a API DeepSeek, mantendo a interface esperada pelo restante do projeto. Essa decisão permitiu preservar a arquitetura do agente e modificar apenas a camada de comunicação com o modelo.

3.4 Camada de Ferramentas

As ferramentas ficam organizadas no diretório `tools/`. Cada ferramenta é uma função Python decorada com `@tool`, o que permite seu registro e disponibilização ao modelo. As principais ferramentas implementadas incluem:

- `listar_colunas()`;
- `descrever_dados()`;
- `contar_valores(coluna)`;
- `filtrar(condicao)`;
- `agrupar_e_agregar(grupo, coluna, funcao)`;
- `correlacao(coluna_a, coluna_b)`;
- `detectar_outliers(coluna)`;

- `gerar_grafico(tipo, colunas)`.

Além das ferramentas obrigatórias, foram adicionadas ferramentas extras, como `diagnosticar_dataset()`, `detectar_correlacoes()` e `gerar_insights()`. Essas ferramentas ampliam a capacidade analítica do agente e permitem respostas mais próximas de uma análise exploratória completa.

4 Metodologia

A metodologia adotada envolveu quatro etapas principais: preparação do *dataset*, implementação das ferramentas, integração com o modelo de linguagem e avaliação por *benchmark*.

4.1 Preparação do *Dataset*

Inicialmente, o dataset de vendas de *e-commerce* foi colocado na pasta `data/` do projeto e configurado em `config.py`. A escolha do dataset considerou os requisitos mínimos de quantidade de registros, número de colunas e presença de variáveis numéricas e categóricas.

4.2 Implementação das Ferramentas

As ferramentas foram implementadas em Python utilizando principalmente Pandas, NumPy, SciPy e Matplotlib. Cada ferramenta foi projetada para retornar um dicionário, permitindo que o resultado fosse serializado, registrado nos logs e enviado de volta ao modelo de linguagem.

A decisão de retornar sempre estruturas em formato `dict` foi importante para padronizar a comunicação entre ferramentas, agente e modelo. Também facilitou a criação do *benchmark*, pois os resultados intermediários podem ser inspecionados e comparados.

4.3 Integração com DeepSeek

A integração com o modelo DeepSeek foi realizada por meio de uma API compatível com o padrão OpenAI. O cliente `llm_client.py` ficou responsável por enviar mensagens, ferramentas e parâmetros ao modelo, além de converter as respostas do modelo para o formato esperado pelo agente.

Essa separação permitiu que o restante do projeto continuasse independente dos detalhes específicos da API. Assim, o agente consome apenas uma interface genérica, composta por mensagens, tools, resposta textual, chamadas de ferramentas, tokens e latência.

4.4 Construção do Benchmark

O benchmark foi estruturado com 30 perguntas, divididas em:

- 10 perguntas factuais simples;
- 15 perguntas analíticas;
- 5 perguntas ambíguas ou inválidas.

Cada pergunta recebeu uma resposta de referência calculada previamente com Pandas. O benchmark executa o agente pergunta por pergunta, registra a resposta obtida, a trajetória, o número de iterações, as chamadas de ferramentas, tokens, latência e custo estimado.

4.5 Métricas Utilizadas

Foram utilizadas as seguintes métricas obrigatórias:

1. Acurácia da resposta final;
2. Taxa de execução bem-sucedida;
3. Número médio de chamadas de ferramentas por pergunta;
4. Latência média;
5. Custo médio por pergunta em dólares e tokens.

Essas métricas permitem avaliar tanto a qualidade da resposta quanto a eficiência operacional do agente.

5 Dificuldades Encontradas e Soluções Adotadas

Durante o desenvolvimento, diversas dificuldades foram encontradas. Esta seção registra os principais problemas e as soluções adotadas, com base no diário técnico do projeto.

5.1 *Benchmark* Marcando Respostas Corretas como Incorretas

Uma das dificuldades mais relevantes ocorreu durante a avaliação automática. Em versões intermediárias, o agente produzia respostas semanticamente corretas, mas o *benchmark* as classificava como incorretas. Isso ocorreu porque os comparadores iniciais eram rígidos e não estavam preparados para respostas em linguagem natural contendo explicações, tabelas ou formatações diferentes.

Um exemplo foi a pergunta sobre a média de desconto por forma de pagamento. O gabarito continha valores como 0,05 e 0,15, enquanto o agente respondia em formato percentual, como 5% e 15%. Embora as respostas fossem equivalentes, o avaliador comparava os números de forma direta e marcava erro.

Outro exemplo envolveu perguntas sobre valores nulos. Para a pergunta sobre a quantidade de valores nulos na coluna `valor_total`, o agente respondeu que a coluna não possuía nenhum valor nulo. O gabarito esperado era o número 0. A resposta estava correta, mas a métrica inicial falhava por não reconhecer expressões como “nenhum” ou “não possui” como equivalentes a zero.

A solução foi refatorar o arquivo `evaluation/metrics.py`, tornando os comparadores mais flexíveis. Foram implementadas funções para extrair múltiplos números, interpretar porcentagens, comparar dicionários numéricos e reconhecer expressões textuais equivalentes a zero. Após esses ajustes, o *benchmark* passou a avaliar corretamente respostas mais completas.

5.2 Respostas Analíticas com Conteúdo Adicional

Outro problema apareceu em respostas analíticas. O agente frequentemente fornecia não apenas o valor solicitado, mas também uma tabela, comentários e

interpretação. Essa característica é positiva para o usuário final, mas dificultou a avaliação automática.

Por exemplo, ao perguntar pela soma de `valor_total` por categoria, o agente retornava uma tabela formatada em *Markdown* e ainda destacava a categoria com maior valor. O *benchmark*, em sua versão inicial, esperava apenas um dicionário numérico. A solução foi ajustar os comparadores para procurar os valores corretos dentro da resposta, sem exigir igualdade textual exata.

6 Resultados Quantitativos

A avaliação final foi realizada com um *benchmark* contendo 30 perguntas. O resultado final indicou 30 acertos, correspondendo a 100% de acurácia geral. A taxa de execução bem-sucedida também foi de 100%, indicando que nenhuma pergunta causou falha ou interrupção do agente.

A Tabela 2 resume as principais métricas obtidas.

Tabela 2 – Métricas finais do benchmark

Métrica	Resultado
Total de perguntas	30
Acertos	30
Acurácia geral	100%
Taxa de execução bem-sucedida	100%
Acurácia factual	100%
Acurácia analítica	100%
Acurácia ambígua/inválida	100%
Tool calls médias	1,77
Latência média	4,71 s
Input tokens totais	186.359
Output tokens totais	9.406
Custo total estimado	US\$ 0,1867
Custo médio por pergunta	US\$ 0,0062

Os resultados indicam que o agente foi capaz de responder corretamente às perguntas propostas no conjunto de avaliação. A média de 1,77 chamadas de ferramenta por pergunta sugere um comportamento eficiente, pois o agente não realizou chamadas excessivas em relação à complexidade das perguntas.

A latência média de 4,71 segundos pode ser considerada aceitável para um sistema interativo baseado em API externa. O custo total estimado de US\$ 0,1867 para 30 perguntas indica que o uso do agente é financeiramente viável em um contexto acadêmico, especialmente considerando que o benchmark completo envolveu múltiplas chamadas ao modelo.

7 Análise Qualitativa

Além das métricas numéricas, foi realizada análise qualitativa das trajetórias do agente. Essa análise é importante porque permite verificar se o agente apenas acertou o resultado final ou se utilizou um caminho coerente para chegar à resposta.

7.1 Trajetória 1: Valores Nulos em `valor_total`

A primeira trajetória analisada corresponde à pergunta: “Quantos valores nulos existem na coluna `valor_total`?”.

A trajetória registrada foi:

1. O agente identificou a necessidade de verificar a estrutura do dataset;
2. Chamou a ferramenta `listar_colunas()`;
3. Confirmou a existência da coluna `valor_total`;
4. Chamou a ferramenta `contar_nulos()`;
5. Recebeu como resultado que não havia valores nulos;
6. Respondeu que a coluna não possuía nenhum valor nulo.

Essa trajetória é relevante porque mostra que o agente não respondeu de forma direta ou por suposição. Ele verificou o *dataset* e utilizou uma ferramenta específica para contar valores nulos. Além disso, essa pergunta foi importante durante o desenvolvimento, pois anteriormente a resposta correta era marcada como incorreta pelo *benchmark* devido à ausência explícita do número 0 em uma versão da resposta.

7.2 Trajetória 2: Média de `valor_total` por Cidade

A segunda trajetória corresponde à pergunta: “Qual é a média de `valor_total` agrupada por cidade?”.

O agente executou a seguinte sequência:

1. Chamou `listar_colunas()` para confirmar os nomes das colunas;
2. Verificou a existência de `cidade` e `valor_total`;

3. Chamou `agrupar_e_agregar()` com grupo `cidade`, coluna `valor_total` e função `mean`;
4. Recebeu as médias por cidade;
5. Gerou uma tabela em *Markdown* e destacou a cidade com maior média.

Essa trajetória demonstra uso adequado de ferramenta analítica. O agente não apenas executou o cálculo, mas também interpretou o resultado, destacando o maior e o menor valor médio. Esse comportamento torna a resposta mais útil para o usuário, embora tenha exigido ajustes no benchmark para aceitar respostas com explicações adicionais.

7.3 Trajetória 3: Correlação de Pearson

A terceira trajetória analisada corresponde à pergunta: “Qual é a correlação de Pearson entre quantidade e `valor_total`?”.

A sequência foi:

1. O agente chamou `listar_colunas()`;
2. Confirmou que `quantidade` e `valor_total` existiam;
3. Chamou a ferramenta `correlacao()` com método Pearson;
4. Recebeu o valor 0,4069;
5. Respondeu com o valor da correlação e uma interpretação qualitativa.

Esse exemplo evidencia a capacidade do agente de realizar análise estatística e produzir interpretação textual. A resposta indicou uma correlação moderada e positiva, mostrando que o agente não se limitou a repetir o número retornado pela ferramenta.

7.4 Análise de Erros Durante o Desenvolvimento

Embora o *benchmark* final tenha atingido 100% de acurácia, ocorreram erros importantes durante o processo de desenvolvimento. O primeiro grupo de erros estava relacionado ao avaliador automático, que marcava respostas corretas como incorretas. A causa principal era a rigidez dos comparadores, que não reconheciam equivalências numéricas, percentuais ou textuais.

O segundo grupo de erros ocorreu na interface *Streamlit*. Problemas como métodos inexistentes, indentação incorreta e tratamento inadequado de JSON exigiram

correções sucessivas. Esses erros mostraram a importância de testar o sistema em diferentes fluxos de uso, não apenas pela CLI.

O terceiro grupo de problemas envolveu a geração de gráficos. A ferramenta de visualização funcionava corretamente, mas gerava arquivos duplicados por utilizar *timestamps*. A solução demonstrou a necessidade de pensar não apenas na execução da ferramenta, mas também na organização dos artefatos gerados.

7.5 Discussão sobre Alucinação

A ocorrência de alucinação foi mitigada pelo uso obrigatório de ferramentas. Nas trajetórias analisadas, o agente demonstrou comportamento consistente: antes de responder, geralmente inspecionava as colunas, executava uma ferramenta e utilizava o resultado calculado para compor a resposta final.

Isso não elimina completamente o risco de alucinação, pois o modelo ainda é responsável por interpretar a pergunta e redigir a resposta. Entretanto, a arquitetura reduz substancialmente esse risco ao forçar a consulta aos dados. Além disso, perguntas ambíguas ou inválidas foram incluídas no *benchmark* justamente para avaliar se o agente reconhece limitações ou solicita esclarecimentos em vez de inventar respostas.

8 Discussão Crítica

8.1 Limitações

Apesar dos bons resultados, o projeto apresenta algumas limitações. O agente foi avaliado apenas em um *dataset* de vendas de *e-commerce*, de modo que a acurácia obtida não garante o mesmo desempenho em outros conjuntos de dados. Além disso, perguntas muito ambíguas ou dependentes de conhecimento externo podem dificultar a interpretação ou a seleção da ferramenta adequada.

A avaliação automática também possui limitações, pois respostas semanticamente equivalentes podem ser apresentadas em formatos diferentes. Embora os comparadores tenham sido aprimorados, ainda existem situações que exigem regras específicas para reconhecimento correto.

8.2 Custo e Latência

A utilização de uma API externa implica custo financeiro e tempo de resposta. No *benchmark* final, o custo médio por pergunta foi baixo e a latência média foi de 4,71 segundos, valores adequados para um contexto acadêmico. Entretanto, aplicações com maior volume de consultas podem exigir otimizações para reduzir chamadas redundantes e melhorar o desempenho.

8.3 Ética e Segurança

O uso de agentes baseados em modelos de linguagem exige atenção à privacidade e ao tratamento de dados. Em aplicações reais, informações sensíveis não devem ser enviadas a serviços externos sem mecanismos adequados de proteção. Também é fundamental proteger credenciais de acesso e evitar sua exposição em repositórios públicos.

8.4 Manutenção e Reprodutibilidade

A estrutura modular do projeto facilita manutenção e futuras expansões. Além disso, a documentação, o código-fonte e a lista de dependências permitem reproduzir o ambiente e executar o sistema de forma consistente.

9 Conclusão

Este trabalho apresentou o desenvolvimento de um agente de análise exploratória de dados em linguagem natural, capaz de responder perguntas em português sobre um dataset CSV utilizando ferramentas Python e um modelo de linguagem DeepSeek. O sistema foi estruturado com separação entre interface, orquestração, ferramentas, dados e avaliação.

O agente implementado atende aos objetivos propostos, pois carrega um dataset, interpreta perguntas em português, escolhe ferramentas autonomamente, executa operações de análise exploratória, registra trajetórias, gera respostas em linguagem natural e produz gráficos quando necessário. A interface *Streamlit* complementou a CLI e tornou o sistema mais acessível, permitindo interação conversacional, upload de arquivos, visualização do dataset, execução de benchmark e consulta a outputs.

A avaliação quantitativa demonstrou desempenho elevado no benchmark final, com 30 acertos em 30 perguntas, 100% de acurácia geral, 100% de taxa de execução bem-sucedida, média de 1,77 chamadas de ferramenta por pergunta, latência média de 4,71 segundos e custo médio estimado de US\$ 0,0062 por pergunta. Esses resultados indicam que a arquitetura proposta foi eficaz para o dataset e conjunto de perguntas avaliados.

A análise qualitativa mostrou que o agente utiliza trajetórias coerentes, geralmente inspecionando colunas antes de executar ferramentas específicas. Também mostrou que o processo de desenvolvimento exigiu ajustes importantes, principalmente no *benchmark*, que inicialmente marcava respostas corretas como incorretas devido à rigidez dos comparadores. A correção dessas métricas foi uma etapa fundamental para avaliar adequadamente respostas em linguagem natural.

Conclui-se que a combinação de LLMs com ferramentas determinísticas é uma abordagem adequada para análise exploratória de dados em linguagem natural. O modelo atua como interface inteligente e orquestrador, enquanto as ferramentas garantem que os cálculos sejam feitos sobre os dados reais. Como trabalhos futuros, recomenda-se ampliar a avaliação para outros *datasets*, adicionar memória de contexto sobre o schema, melhorar a avaliação semântica de respostas e explorar formas de reduzir custo e latência.

Referências

OPENAI. *Function calling and tool use*. Documentação técnica de APIs para modelos de linguagem. Disponível em: <https://platform.openai.com/docs>. Acesso em: 2026.

DEEPSEEK. *DeepSeek API Documentation*. Documentação da API DeepSeek. Disponível em: <https://api-docs.deepseek.com/>. Acesso em: 2026.

MCKINNEY, Wes. *Python for Data Analysis*. 3. ed. Sebastopol: O'Reilly Media, 2022.

PANDAS DEVELOPMENT TEAM. *Pandas documentation*. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: 2026.

STREAMLIT. *Streamlit documentation*. Disponível em: <https://docs.streamlit.io/>. Acesso em: 2026.

FATEC OURINHOS. *Trabalho Final: Agente de Análise Exploratória de Dados em Linguagem Natural*. Material da disciplina de Processamento de Linguagem Natural, 2026.